Advanced Chemical Modification Simulator (CMS)

File: `src/features.py` & `src/model.py` | **API:** `POST /predict`

Theory

Unmodified siRNA degrades rapidly (surviving roughly ~30 minutes in blood serum) and triggers toxic immune responses. Chemical modifications armor the molecule. Our Deep Learning architecture combines biological thermodynamic constants with advanced neural attention to simulate these effects mathematically.

Chemical Modification Baselines

Modification	Tm Impact (°C)	Nuclease Resistance	Immunogenicity (Score)	Source Research
2'-OMe	+0.5 / nt	0.9x	0.3	Setten et al. (2019) / Bramsen (2009)
2'-F	+1.2 / nt	1.0x	0.2	Setten et al. (2019) / Bramsen (2009)
PS Backbone	-0.2 / nt	1.3x	0.5	Setten et al. (2019) / Bramsen (2009)

Critical Rule: Positions 9-12 are the **Ago2 cleavage zone** and must NEVER be heavily modified, or the target mRNA will not be sliced successfully.

> *Source:* Elbashir, S. M. et al. (2001). *Functional anatomy of siRNAs for mediating efficient RNAi.* The EMBO journal.

```
CLEAVAGE_ZONE = set(range(9, 13))  # 0-indexed positions 9, 10, 11, 12
```

Key Calculations & AI Formulas

1. Thermodynamic Free Energy (ΔG)

To calculate whether the strand is loose enough to unzip but stable enough to survive, we use the nearest-neighbor thermodynamic approximation.

Formula: $\Delta G^\circ = \Delta H^\circ - T\Delta S^\circ$

Why we use it (Scientific Context): siRNA functions by unzipping its strands in the RISC complex. If the thermodynamic stability is too high (strongly negative ΔG), the complex cannot unwind the strand. If it is too weak, the strand falls apart before reaching the target. The Nearest-Neighbor model predicts this stability by summing interactions of adjacent nucleotide pairs.

Its exact use in our model: We calculate the local ΔG near the 5'-end, the seed region (positions 2-8), and the sequence as a whole. This is embedded into our 179-feature vector before feeding into the neural network, allowing the AI to understand base-pairing strength.

What output it generates: A continuous scalar float value (e.g., `-9.4 kcal/mol`) representing thermal stability, appended directly into the `feature_vector` tensor.

Research Paper: Mathews, D. H. (2004). *Incorporating chemical modification constraints into a dynamic programming algorithm...* Proc Natl Acad Sci.

```
# Extracting free energy constraints conceptually
def _thermodynamic_features(self, sequence):
    # Free energy impacts (ΔG) mapping
    terminal_stability = calculate_delta_G(sequence.strand[0:4])
    seed_stability = calculate_delta_G(sequence.strand[1:7])
    feature_vector.append(terminal_stability)
```

2. Transformer Self-Attention (The Core Engine)

To properly map how a modification at position 2 chemically clashes with position 18, we bypass legacy RNNs and use an 8-Head Transformer Attention matrix.

Formula: Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V

Why we use it (Scientific Context): Chemical modifications don't exist in a vacuum. A 2'-F at position 3 might interact spatially or electronically with a bulkier 2'-OMe at position 15 across the RNA helix. Traditional models look at sequences sequentially. Self-attention looks at the entire molecule at once, identifying distant 3D structural dependencies inside the sequence.

Its exact use in our model: The sequence and its modifications are passed through 4 layers of Transformer encoders. The Queries (Q), Keys (K), and Values (V) learn to "attend" to specific nucleotide positions. It tells the model *how much* focus position 5 should give to position 12 when predicting overall efficacy.

What output it generates: A contextualized tensor matrix `[batch_size, sequence_length, hidden_dimension]` where every nucleotide's mathematical representation now contains weighted context about every other nucleotide in the sequence.

Research Paper: Vaswani, A. et al. (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems.

```
# src/model.py -> MultiHeadAttention class
attention_scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(sel
attention = torch.softmax(attention_scores, dim=-1)
context_layer = torch.matmul(attention, v)
```

3. Focal Loss for Biological Failure Imbalance

Because highly effective siRNAs are extremely rare, standard Cross-Entropy loss fails. We apply a modulating factor to penalize easy classifications and force learning on hard examples.

Formula: FL(p_t) = -α * (1 - p_t)^γ * log(p_t)

Why we use it (Scientific Context): In siRNA drug discovery, 95% of designed sequences fail or have low efficacy. The dataset is enormously imbalanced towards "ineffective" outcomes. If you train a standard AI on this, it just learns to predict "fail" every time and achieves 95% accuracy without learning any biology.

Its exact use in our model: We use Focal Loss as our primary penalty function during backpropagation. The `(1 - p_t)^γ` term reduces the loss completely for easy examples (the 95% bulk failures) and forces gradients to wildly update weights for the rare 5% highly effective biological "hits". We use `α=0.25` and `γ=2.0`.

What output it generates: A scalar loss gradient tensor representing `Loss.mean()` used by the AdamW optimizer to adjust neural network weights during the `backward()` pass.

Research Paper: Lin, T. Y. et al. (2017). *Focal Loss for Dense Object Detection.* ICCV.

```
# src/model.py -> FocalLoss class
def forward(self, inputs, targets):
    p_t = inputs * targets + (1 - inputs) * (1 - targets)
    focal_weight = self.alpha * (1 - p_t)**self.gamma
    loss = -focal_weight * torch.log(p_t)
    return loss.mean()
```

4. Multi-Task Learning Objective

Efficacy, stability, and immunogenicity are not isolated constraints; they interact. Training them simultaneously on a shared neural backbone improves general accuracy.

Formula: Total Loss = 0.4(L_efficacy) + 0.3(L_stability) + 0.2(L_off_target) + 0.1(L_immunogenicity)

Why we use it (Scientific Context): Biology is heavily multi-variate. A highly efficacious strand might also be highly toxic. Recognizing patterns that govern toxicity forces the neural network's hidden layers to learn more generalized physics and chemistry of the RNA molecule, improving the main goal (efficacy).

Its exact use in our model: Instead of predicting one single score, the neural network terminates in multiple different output heads. Each head calculates its own sub-loss metric. They are summed together using weighted scalars to produce one single backward step.

What output it generates: A final combined scalar loss tensor representing the total multi-objective biological viability, enabling simultaneous gradient updates for multiple scientific endpoints.

Research Paper: Caruana, R. (1997). *Multitask Learning.* Machine Learning, 28(1).

```
# src/model.py -> AdvancedCMSLoss class
total_loss = (
    0.4 * category_loss +
    0.3 * index_loss +
    0.2 * components_loss +
    0.1 * auxiliary_loss
)
```

Over-Modification & Safety Warnings

If modification density exceeds **60%** across the entire strand, or if 2'-Fluoro is repeatedly applied without thermodynamic balancing, the system tensor flags a heavy target binding penalty.

```
# Conceptual Threshold inside feature matrix
if num_modified / len(sequence) > 0.60:
    ago2_affinity_penalty_multiplier = 2.5  # CAUTION: Severely impacte
```